PAYMENT GATEWAY — SYSTEM DESIGN & INTERVIEW REFERENCE V1

Moving money correctly, when every dependency is external and nothing may be lost or duplicated
four-party model · auth & capture · idempotency · ledger · PCI & tokenisation · routing · reconciliation · disputes

1 · WHAT A PAYMENT GATEWAY IS, WHO ELSE IS INVOLVED, AND WHY THIS IS HARD

start here — the participants, the vocabulary, and the three properties that make payments unlike any other system

The definition, and what it is not

A **payment gateway** captures payment credentials securely, turns them into an authorisation request, routes it to whoever can approve it, and reports the outcome. It is the merchant's entry point to the payments network — not the entity that holds or moves the money.

Party	Role
Cardholder	The customer paying
Merchant	Sells the goods; holds a merchant account
Gateway	Captures credentials, routes the request, returns the result
Processor / PSP	Handles the transaction mechanics with the networks. Often bundled with the gateway commercially, but a distinct function
Acquirer	The *merchant's* bank. Holds the merchant account, receives the funds, carries the risk
Card network	Visa, Mastercard, RuPay — routes between acquirer and issuer, sets the rules and the fees
Issuer	The *cardholder's* bank. **Makes the approve-or-decline decision** and bills the customer

This is the four-party model — cardholder, acquirer, network, issuer — and almost every card payment in the world runs on it. Being able to name who actually decides (the issuer) and who actually bears the risk (the acquirer) is the fastest way to show you understand the domain.

Vocabulary the rest of the sheet uses

Authorisation	The issuer checks funds and risk, then **holds** the amount. No money has moved
Capture	The merchant claims the held amount. Usually at dispatch, not at checkout
Void / reversal	Cancelling an authorisation before capture. Cheap and clean
Clearing	Batch exchange of transaction detail between acquirer and issuer
Settlement	The actual movement of funds, typically **T+1 or T+2**
Refund	Returning money *after* capture. A new transaction, not an undo
Chargeback	The cardholder disputes through their issuer. Forcible reversal, plus a fee
PAN	The card number. Never store it
Token	A reference standing in for the PAN
MDR / interchange	What the merchant pays; interchange is the issuer's share of it

The distinction that trips people up. Authorisation is not payment. An approved authorisation is a promise the issuer will honour a claim if made within a window — typically days. Capture is when you claim it. Settlement is when the funds actually arrive. Three different events, three different days.

Why this is harder than it looks

Property	Consequence
Errors are not reversible by a deploy	A double charge is a customer's money and a regulatory matter. There is no rollback — only a compensating refund, and the damage to trust is already done
Every dependency is external	Issuer, network and acquirer are outside your control, on their maintenance windows, with their rate limits and their outages
Timeouts are ambiguous	A timeout tells you **nothing** about whether the customer was charged. This single fact drives most of the design
Correctness is audited	Every rupee must reconcile against the provider and the bank, daily, with an explanation for every break
The rules are regulated	PCI DSS, strong authentication, tokenisation mandates, data residency. Non-compliance is a business risk, not a technical one

The three design commitments everything else follows from. 1 · Every operation that moves money is idempotent, keyed by the caller. **2** · The ledger is append-only and double-entry, so state is derived rather than overwritten. **3** · Nothing is believed until it reconciles against an external source of truth.

Availability target, stated honestly. 99.99% for the API you control, but the end-to-end success rate depends on issuers you do not. Design for graceful failure and clear messaging rather than for a number you cannot deliver alone.

2 · REFERENCE ARCHITECTURE AND THE MONEY TIMELINE

the real-time path on top, the batch path below — and card data never crossing the merchant boundary

CREDENTIAL CAPTURE — the boundary that keeps card data out of the merchant's systems, and therefore out of PCI scope

Customer browser
enters card details into a hosted field / iframe
served by the gateway

Merchant server
never sees the PAN
receives only a token
out of scope by design

Vault / tokenisation
PAN encrypted, HSM-backed
returns a durable token
the only place card data lives

WHY THE BOUNDARY IS DRAWN HERE
The browser posts the card directly to the gateway; the merchant's server receives a token it can safely store.
That single decision moves the merchant to the lightest PCI assessment, because no cardholder data is stored,
processed or transmitted by their systems. Every other design here is downstream of that choice.

TWO KINDS OF TOKEN — and they are not interchangeable
Gateway token — issued by your PSP, meaningful only to them. Simple, but it locks the stored credential to that provider.
Network token — issued by the card network, tied to card and merchant. Survives card reissue, lifts authorisation rates,
and is what regulatory card-on-file tokenisation mandates are built on. Prefer it where available.

REGULATORY CONTEXT WORTH NAMING
PCI DSS governs anyone who stores, processes or transmits card data — the scope you are trying to minimise. **Strong customer authentication** requirements drive 3-D Secure.
In India, the Reserve Bank's card-on-file tokenisation rules mean that from 1 October 2022 no entity in the payment chain other than card issuers and networks may store
card-on-file data — merchants and gateways must use tokens. Confirm the current position with counsel; these rules are amended by notification.

Never stored, anywhere, under any circumstance: the CVV after authorisation, the full PAN outside the vault, or the magnetic-stripe / chip data. There is no legitimate reason and no exception.
What you may store: the token, the last four digits, the card brand, the expiry, and an issuer BIN range for routing. That is enough to build every customer-facing feature people ask for.

AUTHORISATION — THE REAL-TIME PATH · budget ≈ 2–3 s end to end, and most of it is not yours

1 · Payment API
idempotency key
validate, persist intent
record BEFORE calling out
~10 ms

2 · Risk / fraud
velocity, device, BIN
score → allow/challenge/deny
must be fast or bypassed
~50 ms

3 · 3-D Secure
frictionless or challenge
shifts liability to issuer
customer-dependent latency
0 s … minutes

4 · Routing engine
choose acquirer / PSP
cost, auth rate, health
circuit breaker per route
~5 ms

5 · Acquirer
merchant's bank
forwards to the network
outside your control
~300 ms

6 · Card network
Visa / Mastercard / RuPay
routes to the issuer
very reliable, not free
~200 ms

7 · Issuer decides
funds, limits, own risk model
approve / decline / refer
the actual decision point
~500 ms, sometimes far worse

8 · THE RESPONSE, AND WHAT YOU DO WITH IT
Approved — write the ledger entry and the state transition in **one transaction**, with the outbox event. Only then respond.
Declined — a normal outcome, not an error. Map the issuer's reason code to something the customer can act on.
Timeout — the hard case. Not a failure. Move to UNKNOWN and resolve by query, never by retrying blindly.

WHERE THE LATENCY ACTUALLY GOES — and why an aggressive timeout is dangerous here
Your own components account for well under 100 ms of a roughly 2-second round trip. **Everything else belongs to the acquirer, the network and the issuer** — and issuer latency varies by bank, by time of day and by whether they are having a bad afternoon. Optimising your side buys single-digit percentages; choosing a better route buys far more.
Set the client timeout generously — 30 s or more. A short timeout does not cancel the transaction upstream; it only destroys your knowledge of what happened, and manufactures UNKNOWN states.

THE MONEY TIMELINE — four distinct events, on four different clocks. Conflating them is the most common domain mistake.
AUTHORISE (t=0, seconds) — the issuer holds the amount against the customer's limit. **No money has moved.** The hold expires on its own after days if never captured.
CAPTURE (t = hours to days) — you claim the held amount, normally at dispatch. Capturing at checkout for goods you have not shipped is a refund liability and, in some sectors, a compliance problem.
CLEAR (t+1, batch) — acquirer and issuer exchange transaction detail. This is where fees are computed and where your figures and theirs must agree.
SETTLE (t+1 to t+2) — funds actually reach the merchant's account, net of fees. **A payment your system shows as successful is not money in the bank until this happens.**

THE BATCH AND ASYNCHRONOUS SIDE — where correctness is actually established, and where most of the engineering lives

Ledger
double-entry, append-only
immutable journal
the source of truth

Outbox → Kafka
state change + event
in one transaction
no dual writes

Webhook dispatcher
signed, retried, ordered
merchant-facing
replayable on demand

Status resolver
sweeps UNKNOWN
queries the PSP
stops stuck payments

Reconciliation
settlement file vs ledger
daily, three-way
every break investigated

Payouts
merchant settlement
net of fees and holds
its own approval path

Dispute handling
chargeback lifecycle
evidence, deadlines
deadlines are hard limits

THE PRINCIPLE THAT SEPARATES A PAYMENT SYSTEM FROM AN ORDINARY ONE
Nothing is considered true because your service returned success. It is true when it appears in the provider's
settlement file and your ledger agrees with it. Every other system treats its own database as authoritative;
a payment system treats an external file as authoritative and its own database as a claim awaiting confirmation.

CONSISTENCY CHOICES — CP almost everywhere, and deliberately so
CP: the ledger, payment state, idempotency records, balances. Refuse the write rather than risk divergence — an unavailable API is recoverable, a wrong balance is not.
AP: merchant dashboards, transaction search, analytics, reporting. Reading a figure a few seconds stale harms nobody.
This is the one domain where defaulting to CP is correct, and where "eventually consistent" needs justification rather than being assumed.

READING THE DIAGRAM
The top band exists to keep card data out of your systems. The middle band is two seconds long and mostly belongs to other companies. The bottom band is where correctness is actually established — and it is the part that is usually missing from a payment architecture diagram, which is why so many of them describe systems that cannot prove they are right.

3 · THE PAYMENT STATE MACHINE

including the state people omit

```
CREATED
  │
  ├─→ AUTHORISED ──→ CAPTURED ──→ SETTLED
  │       │             │
  │       │             ├─→ REFUNDED
  │       │             ├─→ PARTIALLY_REFUNDED
  │       │             └─→ DISPUTED ─→ CHARGEBACK_LOST
  │       │                          └─→ CHARGEBACK_WON
  │       ├─→ VOIDED          // cancelled pre-capture
  │       └─→ AUTH_EXPIRED    // hold lapsed, not captured
  │
  ├─→ DECLINED        // issuer said no — a normal outcome
  ├─→ FAILED          // our side or a validation error
  │
  └─→ UNKNOWN  ◄── timeout / no response
          │
          └─→ resolved by query or reconciliation
              into one of the states above
```

`UNKNOWN` is the state most designs are missing, and it is the one that causes double charges. It is not a failure and must never be treated as one. A payment in this state has an unresolved external truth, and the only valid actions are **query the provider** or **wait for reconciliation** — never retry, and never tell the customer it failed.

Rules for the machine

- **Transitions are append-only events**, not column updates. The history is what an auditor asks for
- **Only forward transitions are legal.** A `CAPTURED` payment never returns to `AUTHORISED`
- **Out-of-order webhooks are normal** — ignore any transition older than current state
- **Every terminal state has an owner** and a defined customer-facing message
- **Alert on anything stuck** beyond its expected window — this is how stranded payments are found before customers find them

`AUTH_EXPIRED` is worth modelling explicitly. Authorisation holds lapse after a period set by the network and issuer. If you capture late and the hold has gone, the capture fails and the customer may already have the goods. Track the expiry and act before it.

4 · IDEMPOTENCY AND THE AMBIGUOUS TIMEOUT

the single most important mechanism on this sheet — everything that moves money depends on it

The governing fact, stated once. When a payment call times out, **four things are equally possible:** the request never arrived; it arrived and was declined; it arrived and succeeded but the response was lost; or it is still being processed. Your system cannot distinguish them. Every mechanism below exists because of that.

The idempotency contract

```
// caller generates the key when intent is formed,
// and reuses it for every retry of the SAME intent
POST /v1/payments
Idempotency-Key: 01J8F3K2M9Q

// server, in one transaction:
INSERT INTO idempotency(key, request_hash, state)
  VALUES (?, ?, 'IN_PROGRESS')
  ON CONFLICT (key) DO NOTHING
// 0 rows → this key was seen before

  ├─ completed?   → return the ORIGINAL response
  ├─ in progress? → return 409, tell caller to poll
  └─ new?         → process, store the result
```

Return the original response, not an error. A repeat with the same key must return the first result. Returning a conflict makes the caller believe it failed — and they will retry again, which is precisely the loop you are trying to prevent.

Bind the key to the request body. Store a hash of the payload; if the same key arrives with a different amount, reject it. Otherwise a client bug can turn one key into a way of overwriting an earlier payment's identity.

Retry policy — the rules that prevent double charges

Situation	Correct action
Connection refused request provably never landed	Safe to retry with the same key
Timeout after send	**Do not retry blindly.** Retry only with the same idempotency key, or query the provider for the status of that key
HTTP 5xx	Retry with the same key, with exponential backoff and jitter
HTTP 4xx	Do not retry. It is deterministic and will fail identically
Explicit decline	Do not retry automatically. A different card or method is a customer decision

The three-layer defence. 1 · Your idempotency key stops your own retries duplicating. **2** · The PSP's idempotency key stops your calls duplicating at their end — pass one, always. **3** · Daily reconciliation catches anything that slipped through both. Any one layer alone is insufficient.

Key lifecycle, which is usually forgotten

- **Generated at intent**, in the client or at the start of the flow — never at the retry site, which would produce a fresh key each attempt and defeat the mechanism entirely
- **Carried end to end**, including to the PSP
- **Retained longer than the longest plausible retry window** — days, not minutes. Delete too early and a late retry is processed as new
- **Scoped per merchant**, so one merchant's keys cannot collide with another's

Exactly-once does not exist across systems. What you build is at-least-once delivery with idempotent processing, which produces the same observable outcome — and saying so precisely is a stronger answer than claiming a guarantee that is not achievable.

5 · THE LEDGER — DOUBLE-ENTRY, IMMUTABLE, AND WHY MONEY IS AN INTEGER

how a payment system proves it is right, rather than asserting it

Double-entry, in one paragraph

Every movement is recorded as balanced entries across two or more accounts. Debits equal credits, always. A balance is not a stored number that gets updated — it is the **sum of the entries**. That is what makes the system auditable and self-checking.

```
// customer pays ₹2,499.00 — captured
journal_entry: je_8831    ts: 2026-09-01T10:14:22Z
  DEBIT   merchant_receivable   249900
  CREDIT  customer_payable      249900

// gateway fee 2% recognised at capture
journal_entry: je_8832
  DEBIT   merchant_receivable     4998
  CREDIT  fee_revenue             4998

// settlement lands T+1, net of fees
journal_entry: je_9104
  DEBIT   bank_settlement       244902
  CREDIT  merchant_receivable   244902

// invariant, checked continuously:
SUM(debits) == SUM(credits)  — always, no exceptions
```

Why this is worth the effort. When a figure is disputed, you can show every entry that produced it and when it arrived. An overwritten balance column can only tell you what it says now — which is exactly the wrong answer to give an auditor or a regulator.

Money representation — non-negotiable rules

Integer minor units	₹2,499.00 is stored as `249900`. **Never a float** — binary floating point cannot represent 0.1 exactly, and rounding errors in money are real losses that accumulate
Always with a currency	An amount without a currency code is meaningless. Store them together, as one value object
Minor units differ	Two decimal places is not universal — some currencies have none, some have three. Do not hard-code 100
Rounding is a policy	Decide it once, write it down, apply it consistently, and record which entry absorbed the remainder when splitting
FX is a dated fact	Store the rate and the timestamp used with the transaction. Recomputing later with today's rate produces a different answer and an unexplainable break

Design properties that follow

- **Append-only.** No UPDATE, no DELETE on journal entries. A correction is a new reversing entry, so the mistake and the fix are both visible
- **Idempotent by construction** — a journal entry carries the idempotency key that produced it, so replay cannot double-post
- **CP, single-writer per account** — this is exactly where strong consistency is worth its cost
- **Partition by time**, and keep the history. It will be asked for
- **Continuous invariant checks** — debits equal credits, no negative balances where prohibited, no orphan entries. Alert on violation immediately

Do not put the ledger in an eventually-consistent store. Redis, a search index or a replica are all fine for *reading* balances on a dashboard. The authoritative entries belong in a transactional database with real isolation — this is the clearest case on any of these sheets for choosing CP.

6 · PCI SCOPE & TOKENISATION

designing the problem away

PCI DSS applies to any system that **stores, processes or transmits** cardholder data. The winning strategy is not to comply harder — it is to arrange the architecture so almost nothing is in scope.

How scope is minimised

Approach	Effect
Hosted fields / iframe	The card input is served by the provider; the browser posts the PAN directly to them. **The merchant server never receives it**, qualifying for the lightest assessment
Full redirect	Simplest scope of all, at the cost of losing control of the checkout experience
Direct API	Card data crosses your servers. Full assessment, annual audit, network segmentation. **Only worth it if you are the processor**

Client-side scripts are now the attack surface that matters. Recent PCI guidance places specific emphasis on authorising and monitoring scripts on the payment page, because skimming attacks target the browser rather than the server. An iframe helps, but a compromised parent page is still a risk — inventory your scripts and enforce integrity.

Never stored, under any circumstances

- **CVV / CVC after authorisation** — prohibited outright, no exceptions for convenience
- **Full PAN outside a vault**, and never in logs, traces, error messages, analytics or support tickets
- **Track or chip data** post-authorisation

What you may safely keep: the token, last four digits, brand, expiry and BIN range. That is sufficient for saved cards, receipts, routing and support — every customer-facing feature people ask for.

Redaction must be enforced, not requested. Card numbers reach logs through error objects, request dumps and third-party SDKs. Filter at the logging library and scan for PAN patterns in your log pipeline — a convention that relies on developers remembering will fail.

7 · AUTHENTICATION & FRAUD

the trade-off nobody escapes

3-D Secure — what it actually does

The issuer authenticates the cardholder before authorisation. Modern implementations pass rich device and transaction data so most sessions are **frictionless** — approved with no customer interaction — and only riskier ones are challenged.

The commercial point is liability. A successfully authenticated transaction generally shifts fraud liability from the merchant to the issuer. That is why merchants adopt it even though every added step costs conversion.

The tension, stated honestly

More friction	Less fraud, more liability protection, **and measurably fewer completed checkouts**
Less friction	Higher conversion, more fraud losses and more chargebacks

Route by risk, not by rule. Challenge the risky minority and let the rest through. A blanket policy either loses revenue on safe transactions or absorbs fraud on dangerous ones — and where regulation mandates strong authentication, exemptions usually exist for low-value or low-risk cases and are worth using.

Fraud signals worth having

- **Velocity** — attempts per card, per device, per IP, per address in a window
- **Device fingerprint** and session behaviour
- **BIN intelligence** — issuer country versus billing and shipping country
- **Mismatch signals** — billing/shipping distance, new address on a first order
- **Card testing** — many small authorisations across many cards from one source

The cost of a false positive is invisible and larger than it looks. A declined genuine customer rarely complains — they leave. Track the decline rate as carefully as the fraud rate, and sample declined transactions for review, or the model will quietly optimise itself into blocking real revenue.

8 · ROUTING & RESILIENCE

every dependency is someone else's

Once volume justifies more than one acquirer or PSP, **which route a transaction takes becomes a design decision** — and one with direct revenue consequences.

Why route at all

Authorisation rate	The same card can be approved by one acquirer and declined by another. Small percentage differences are large revenue differences
Cost	Fees vary by route, card type and geography
Redundancy	Providers have outages. A single integration means their downtime is your downtime
Local acquiring	Domestic routes usually approve better and cost less than cross-border ones
Method coverage	Cards, bank transfers, wallets and real-time rails rarely come from one provider

Making it safe

- **Health-aware routing** — track success rate and latency per route and shift traffic away from a degrading one automatically
- **Circuit breaker per provider**, so a failing route fails fast instead of consuming your capacity
- **Cascading retry with care** — retrying a *declined* transaction on another route is sometimes legitimate and sometimes prohibited by scheme rules. Know which, and never cascade on a timeout
- **Provider-specific idempotency keys** so a cascade cannot double-charge
- **An abstraction layer** so adding a provider is configuration, not a rewrite — but expect their semantics to differ more than their APIs suggest

Cascading on an ambiguous outcome is how double charges happen at scale. If route A timed out, you do not know it failed. Resolve A's status first. Only a **definitive decline** is safe to re-route, and even then only where the scheme permits it.

Measure authorisation rate per route, per BIN, per hour. It is the single most valuable metric in a payments business, and it moves without anything in your code changing.

9 · TECHNOLOGY CHOICES

and the alternative

Need	Choice, why, and what it costs
Ledger	**PostgreSQL.** Real transactions, constraints, serializable isolation where needed. *Cost:* single-writer scaling. *Alternative:* distributed SQL when write volume genuinely exceeds one node
State & idempotency	**The same database.** Keeping the record and its effect in one transaction is the whole mechanism. *Never* a separate store — that reintroduces the dual-write problem
Events	**Kafka** for replay and multiple consumers. *Cost:* operational weight. *Alternative:* an outbox table with a poller, which is enough at moderate volume
Vault	**The provider's vault**, or an HSM-backed service. *Cost:* a coupling. *Alternative:* building your own — only if you are the processor
Read models	**A replica or search index** for dashboards and transaction search. Keeps reporting load off the ledger
Reconciliation	**Columnar or warehouse** for matching millions of rows daily. *Alternative:* the transactional database, until the job stops fitting its window
Secrets & keys	**Managed KMS / HSM** with rotation. Non-negotiable for key material

The honest summary. A payment system is not technically exotic — it is a transactional database, a queue and a great deal of discipline. The difficulty is in the invariants, the failure handling and the reconciliation, none of which a technology choice solves for you.

10 · RECONCILIATION, SETTLEMENT AND DISPUTES

the unglamorous half, where correctness is proved and where most payment engineering time actually goes

Three-way reconciliation, daily

```
   your ledger        ↔    PSP report    ↔    bank statement
   (what you think)       (what they say)     (what arrived)

// every transaction must appear in all three,
// with the same amount, on the same date
```

Break type	Usual cause
In ours, not theirs	An `UNKNOWN` we optimistically marked successful. **The dangerous one**
In theirs, not ours	A response we lost — the customer was charged and we do not know
Amount mismatch	Fees, FX rounding, partial capture
Date mismatch	Time zone or cut-off boundary, not a real break
Duplicate	An idempotency failure. Investigate the root cause, never just net it off

Reconciliation is not a report — it is a control. Every break needs an owner, an age and a resolution. Unexplained breaks accumulating silently is the state in which a payment business discovers a problem months late and cannot say how much money it involves.

Automate the matching, escalate the exceptions. The great majority match on reference and amount. Engineering effort belongs in classifying the remainder and in making each class self-explanatory to the operations team.

Settlement and payouts

- **Funds arrive net** of fees and any withheld reserve — the gross figure your system shows is never what lands
- **Payout schedule** is a product decision: daily, weekly, or on demand for a fee
- **Rolling reserve** — a percentage held against future chargebacks, released on a schedule. Model it explicitly or merchant balances will not reconcile
- **Payout is itself a payment** — idempotent, ledgered, reconciled, and with its own approval path for large amounts
- **Fee computation must be reproducible** — record the rate applied and the basis, not just the result

The dispute lifecycle

Stage	What happens
Retrieval request	The issuer asks for information before a formal dispute
Chargeback	Funds are pulled back, plus a fee. **You are debited first and argue afterwards**
Representment	You submit evidence — delivery proof, authentication result, terms accepted, communication history
Pre-arbitration / arbitration	Escalation, with rising costs on both sides

Deadlines are hard limits, not targets. Miss the response window and the dispute is lost automatically regardless of the merits. Build the timer into the system, alert well before expiry, and treat it as an availability concern rather than a support task.

Watch the chargeback ratio. Card networks impose monitoring programmes above threshold ratios, with fines and — at the extreme — loss of the ability to accept cards. It is one of the few engineering metrics that can end a business line.

11 · API SPECIFICATION

the contract a merchant integrates against — including the parts that make it safe to retry

Method	Endpoint	Purpose & status codes
POST	`/v1/tokens`	Exchange card data for a token. **Called from the browser**, never the merchant server · **201**
POST	`/v1/payments`	Create and authorise · **201** · **`Idempotency-Key` required** · 402 declined · 409 key in progress
GET	`/v1/payments/{id}`	Current state · **200** · the resolution path for `UNKNOWN`
POST	`/v1/payments/{id}/capture`	Claim an authorised amount; supports partial · **200** · 409 if already captured or expired
POST	`/v1/payments/{id}/void`	Cancel before capture · **200** · 409 if captured
POST	`/v1/refunds`	Refund fully or partially · **201** · `Idempotency-Key` required · 422 if exceeds captured
GET	`/v1/payments`	List and search · **200** · cursor pagination · served from a read model
POST	`/v1/customers/{id}/methods`	Save a token against a customer for repeat use · **201**
GET	`/v1/disputes/{id}`	Dispute state and the evidence deadline · **200**
POST	`/v1/disputes/{id}/evidence`	Submit representment evidence · **202**
GET	`/v1/settlements/{date}`	Settlement summary and the per-transaction breakdown · **200**
GET	`/v1/balance`	Available, pending and reserved, from the ledger · **200**

Note 402 for a decline. A declined payment is a successful API call with a business outcome — not a 500. Conflating the two makes every merchant's error handling wrong, and makes your own alerting meaningless.

Create a payment

```
POST /v1/payments
Authorization: Bearer sk_live_…
Idempotency-Key: 01J8F3K2M9Q
{
  "amount":   { "value": 249900,
                "currency": "INR" },
  "method":   { "type": "card",
                "token": "tok_9f21…" },
  "capture":  false,          // authorise only
  "reference": "order_1183", // merchant's id
  "customer": "cus_44",
  "metadata": { "channel": "web" }
}

HTTP/1.1 201 Created
{
  "id": "pay_8f3c1a",
  "status": "AUTHORISED",
  "amount": { "value": 249900, "currency": "INR" },
  "captured": { "value": 0, "currency": "INR" },
  "authExpiresAt": "2026-09-08T10:14:22Z",
  "method": { "brand": "visa", "last4": "4242" },
  "riskScore": 12,
  "createdAt": "2026-09-01T10:14:22Z"
}
```

Returning `authExpiresAt` is a small thing that prevents a common failure. It tells the merchant when the hold lapses, so late captures can be scheduled or abandoned deliberately rather than discovered as an error.

A decline, and an error

```
HTTP/1.1 402 Payment Required
{
  "id": "pay_8f3c1b",
  "status": "DECLINED",
  "declineCode": "insufficient_funds",
  "issuerMessage": "Do not honour",
  "retryable": false,
  "requestId": "req_77a2"
}

HTTP/1.1 409 Conflict
{
  "code": "IDEMPOTENCY_IN_PROGRESS",
  "message": "Original request still processing",
  "retryAfter": 2,
  "requestId": "req_77a3"
}
```

The `retryable` flag earns its place. Merchants cannot be expected to maintain a mapping of every issuer decline code. Telling them plainly whether another attempt could succeed prevents both pointless retries and abandoned recoverable payments.

Webhooks — both directions

- **Signed** with a shared secret and a timestamp; reject anything outside a small clock window
- **At-least-once and unordered** — deduplicate by event id and ignore stale transitions
- **Retried with backoff** over hours, then parked for manual replay
- **A replay endpoint** so a merchant can recover after their own outage
- **Never the only channel** — the payment status endpoint must always be authoritative

12 · PRODUCTION PROBLEMS

what actually goes wrong, and the answers that show you have run one of these

Problem	What you see	Answer
Double charge	Customer billed twice; discovered by them, not you	Idempotency at both layers, never cascade on timeout, daily reconciliation
Stuck in UNKNOWN	Payments neither succeed nor fail; support tickets accumulate	A status-resolver sweep with backoff, plus an age alert
PSP outage	Authorisation rate falls off a cliff for one route	Health-aware routing, circuit breaker, failover route, honest customer messaging
Issuer degradation	One bank's cards timing out; overall rate looks fine	Slice authorisation rate by BIN and issuer, not just in aggregate
Card testing attack	Thousands of small authorisations, tiny approval rate, fees mounting	Velocity limits per card, device and IP; CAPTCHA; alert on the decline-rate spike
Reconciliation drift	Breaks growing daily, nobody owning them	Break ageing report, an owner per class, and alerting on unexplained totals

Problem	What you see	Answer
Late capture failure	Capture rejected because the authorisation expired; goods already shipped	Track `authExpiresAt`, alert before expiry, re-authorise where permitted
Refund exceeds capture	Over-refunding through partial refunds	Enforce the invariant in the ledger, not only in the API layer
FX rounding drift	Small persistent breaks in multi-currency settlement	Store the rate and timestamp with the transaction; never recompute later
Webhook storm	A merchant's endpoint fails; your queue backs up behind it	Per-merchant queues and rate limits so one cannot affect another
Chargeback deadline missed	Disputes lost by default, without ever being reviewed	Deadline tracked as a first-class field with escalating alerts
PAN in logs	Card numbers in an error trace, discovered during audit	Redaction in the logging library, plus pattern scanning in the pipeline

What breaks first — the bottleneck ladder

#	Limit
1	**The PSP's rate limit.** External, and you cannot scale past it. Negotiate headroom before a sale, not during one
2	**Issuer latency** dragging p99 past your client timeout — which manufactures `UNKNOWN` states faster than the resolver clears them
3	**Ledger single-writer throughput** per account. The only one you can fix with your own infrastructure
4	**Reconciliation runtime** exceeding its window, so breaks are found a day late and compound
5	**Webhook delivery backlog** when a large merchant's endpoint degrades

Note that only one of the five is yours. Payment capacity planning is mostly about other people's limits, which is why the conversation with the provider matters more than the autoscaling configuration.

What I would monitor above all

- **Authorisation rate**, sliced by route, BIN and issuer — it moves without any deploy
- **Payments in `UNKNOWN`**, and their age distribution
- **Unexplained reconciliation breaks**, and their total value
- **Duplicate detection rate** — a rising number means idempotency is being tested and holding
- **Time to dispute deadline** for anything open

The one alert that matters most is a fall in authorisation rate. It is the earliest signal of a provider problem, an issuer problem, a fraud-rule regression or an attack — and it is a revenue metric, so it gets attention.

13 · READINESS CHECKLIST

before this system touches real money

- ☐ **Card data never reaches your servers** — hosted fields or redirect
- ☐ **CVV never stored**; PAN only inside the vault
- ☐ **PAN redaction enforced** in the logging library and scanned for in the pipeline
- ☐ **Tokenisation approach chosen**, and network tokens used where available
- ☐ **Applicable regulations confirmed with counsel**, with the date recorded
- ☐ **Idempotency key required** on every money-moving endpoint
- ☐ **Key bound to a request hash**; mismatches rejected
- ☐ **Repeat requests return the original response**, not an error
- ☐ **Idempotency records retained** beyond the longest retry window
- ☐ **Idempotency key passed through to the provider** as well
- ☐ **`UNKNOWN` is a modelled state**, never mapped to failure
- ☐ **Status-resolver sweep runs**, with an age alert on stuck payments
- ☐ **No blind retry after a timeout**; no cascade on an ambiguous outcome
- ☐ **Client timeouts set generously** — short ones manufacture unknowns
- ☐ **Ledger is double-entry and append-only**; corrections are reversing entries
- ☐ **Debits equal credits**, checked continuously and alerted on
- ☐ **Money is integer minor units** with an explicit currency
- ☐ **FX rate and timestamp stored** with the transaction
- ☐ **Rounding policy written down** and applied consistently
- ☐ **Refund cannot exceed captured** — enforced in the ledger
- ☐ **Authorisation expiry tracked**, with alerts before it lapses
- ☐ **Capture at dispatch**, not at checkout, unless the domain requires otherwise
- ☐ **Declines returned as 402** with an actionable code, not as errors
- ☐ **Webhook signatures verified**; events deduplicated and order-tolerant
- ☐ **Outbound webhooks signed, retried and replayable**; per-merchant isolation
- ☐ **Status endpoint is authoritative**, never the webhook alone
- ☐ **Three-way reconciliation runs daily**, with every break owned and aged
- ☐ **Duplicates in reconciliation investigated**, never silently netted off
- ☐ **Fees and reserves modelled** so merchant balances reconcile
- ☐ **Payouts are idempotent, ledgered and approved** above a threshold
- ☐ **Dispute deadlines tracked** with escalating alerts
- ☐ **Chargeback ratio monitored** against scheme thresholds
- ☐ **Authorisation rate sliced by route, BIN and issuer**
- ☐ **Circuit breaker and failover route** per provider, tested
- ☐ **Velocity limits** against card testing, with decline-spike alerting
- ☐ **Decline rate watched as closely as fraud rate**; declines sampled for review
- ☐ **Runbooks** for double charge, stuck payment, PSP outage, reconciliation break
- ☐ **Failure paths tested** by deliberately timing out mid-authorisation under load

The one that summarises the sheet: **kill the network mid-authorisation, then answer three questions — was the customer charged, what does your database say, and what will the settlement file say tomorrow?** If those three answers can disagree without anything alerting, the system is not ready to take money.